2. Solution

CSA06 – Design and Analysis of Algorithms

CO1 Assessment Tool 1: Scalability Audit of "ShopSphere"

Student Name: E.C. Jithendra Kumar Reddy Reg. No.: 192472180

Assumptions used throughout

- n = 100,000 today and n = 1,000,000 within a year.
- All logarithms are base 2.
- Server speed is R = 10^8 basic operations per second. The exponent is blank in the question paper, so I also give the 10^9 results in part (g).
- The nightly window is 10 minutes = 600 seconds, which allows at most R × 600 = 6 × 10^{10} operations at 10^8 per second.
- One basic operation costs one unit of time.

(a) Understanding the Problem

Module	Input size	Basic operation	Where it occurs	Expected growth of its count
M1 Duplicate detector	n = number of products	The test title = title AND seller = seller	Inside the inner loop, once for every pair (i, j)	Quadratic. Ten times more products means about 100 times more tests.
M2 API rate limiter	n = size of the request log	Audit(Request[j]) (given in the question)	Inside the inner loop, which runs i times per pass while i doubles	Linear. The passes grow geometrically, so the last pass dominates.

M3 Product search (S1 and S2)	n = products in the catalogue or index	Comparison of the search key with an entry	S1: each step of the scan. S2: each probe of the middle entry.	S1 grows linearly. S2 grows logarithmically, adding one comparison per doubling of n.
M4 Recommendation engine	n = size of the customer–product data	Merging one element of the partial results	The linear merge step, which is the n term in the recurrence	Current design quadratic, redesign n log n.
M5 Sales analytics and report generator	n = number of sales records	Comparison made while sorting at each level. For the report generator, the step counted by the $3n^2$ term.	At every level of the recursion, the sort costs n log n.	Analytics grows like n $\log^2$ n. The report generator grows quadratically.

Justification. In each case the chosen operation is the one repeated most often, and everything else in the module happens at most a constant number of times per execution of it. Counting it therefore captures the growth of the whole module. For M4 and M5 the operation is read off the recurrence: the non-recursive term f(n) is the work done at one level of the recursion.

(b) Analysing the Iterative Modules

M1: Duplicate-Listing Detector

Exact count. For each i from 0 to n − 2, the inner loop runs for j = i + 1 to n − 1, which is (n − 1 − i) times. The total number of comparisons is

$C_1(n) = \Sigma_{i=0}^{n-2} (n - 1 - i) = (n - 1) + (n - 2) + ... + 2 + 1$

This is the sum of the first n − 1 integers, so

$C_1(n) = n(n-1)/2$

The count does not depend on the data, because the loops always run to completion. Best, average and worst cases are all the same.

Time complexity: $n(n-1)/2 = n^2/2 - n/2$, which is $\Theta(n^2)$.
Space complexity: only the counters i and j are needed, and duplicates are flagged in place, so the extra space is $\Theta(1)$.

n	Comparisons n(n − 1)/2
100,000	4,999,950,000 (about 5.0×10^9)
1,000,000	499,999,500,000 (about 5.0×10^{11})

Meaning for ShopSphere. Growing the catalogue 10 times makes the nightly job 100 times heavier. At 10^8 operations per second it takes about 50 seconds today and about 5,000 seconds (83 minutes) next year.

M2: API Rate Limiter

Exact count. The outer loop runs with i = 1, 2, 4, ..., 2^(m−1), continuing while i < n. Taking i = 2^t, the condition 2^t < n gives $t \le \lceil \log n \rceil - 1$, so there are $m = \lceil \log n \rceil$ passes. In the pass with value i, the inner loop calls Audit exactly i times. The total is a geometric series:

$A(n) = 1 + 2 + 4 + \ldots + 2^{(m-1)} = 2^m - 1$

$A(n) = 2^{\lceil \log n \rceil} - 1$

Since $2^m \ge n$, we have $A(n) \ge n - 1$. Since $2^{(m-1)} < n$ gives $2^{(m-1)} \le n - 1$, we have $2^m \le 2n - 2$ and so $A(n) \le 2n - 3$. Therefore

$n - 1 \le A(n) \le 2n - 3$, which gives $A(n) = \Theta(n)$.

Is the team member's claim correct? No. The reasoning has a mistake. The inner loop does not run "up to n times" in every pass. It runs i times, and i is small in the early passes: 1, 2, 4, 8, and so on. Multiplying the number of passes (log n) by the largest inner-loop count (n) overestimates the work. The correct sum is geometric and is dominated by its last term, so it is

a constant multiple of n, not n log n. It is technically true that A(n) = O(n log n), but that bound is not tight, so the claim "Θ(n log n)" is false. The tight bound is Θ(n).

Verification for n = 1,024. The passes use i = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (the loop stops because 1,024 < 1,024 is false), which is 10 passes = log 1,024. The number of Audit operations is

1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1,023

The formula gives $2^{10} - 1 = 1{,}023$. ✓ This equals n − 1, as expected for a power of 2. The incorrect claim would predict about n log n = 1,024 × 10 = 10,240, which is ten times too high. Running the loop in code (Python_Code.py) also gives 1,023.

Space complexity: Θ(1).

n	Audit operations 2^⌈log n⌉ − 1	Wrongly claimed n log n
100,000	131,071	about 1.66×10^6
1,000,000	1,048,575	about 1.99×10^7

Meaning for ShopSphere. M2 is cheap. Going from 100,000 to 1,000,000 requests costs only about 8 times more, and about a million operations is roughly 0.01 seconds. If the team member's claim had been accepted, the module would have been overestimated by a factor of about 19.

(c) Solving the S2 Recurrence by Backward Substitution

Recurrence: T(n) = T(n/2) + 1, T(1) = 1. Take n = 2^k, so that halving always gives a whole number.

Step 0: T(n) = T(n/2) + 1

Step 1: T(n/2) = T(n/4) + 1 so T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2

Step 2: T(n/4) = T(n/8) + 1 so T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3

Step 3: T(n/8) = T(n/16) + 1 so T(n) = T(n/16) + 4

...

Step i: T(n) = T(n/2^i) + i

The pattern after i steps is T(n) = T(n/2^i) + i. The recursion stops at the base case, n/2^i = 1, which means 2^i = n and i = log n. Substituting:

T(n) = T(1) + log n = 1 + log n

Closed form: $T(n) = \log_2 n + 1$ for n = 2^k.
For any n (using ⌊n/2⌋ in the recurrence): $T(n) = \lfloor \log_2 n \rfloor + 1$.

Proof for general n by induction. Base: T(1) = 1 = ⌊log 1⌋ + 1 ✓. Step: for n ≥ 2, assuming the formula for smaller values,
T(n) = T(⌊n/2⌋) + 1 = (⌊log ⌊n/2⌋⌋ + 1) + 1 = (⌊log n⌋ − 1) + 2 = ⌊log n⌋ + 1 ✓

Check: T(8) = T(4) + 1 = T(2) + 2 = T(1) + 3 = 4, and log 8 + 1 = 4 ✓. For n = 1,000,000: ⌊19.93⌋ + 1 = 20 comparisons.

Implementation	Time complexity	Space complexity
Iterative S2	Θ(log n)	Θ(1): only low, high and mid are kept
Recursive S2	Θ(log n)	Θ(log n): each call adds a stack frame, and the depth is ⌊log n⌋ + 1 (20 frames at n = 1,000,000)

Meaning for ShopSphere. Each time the catalogue doubles, S2 needs just one more comparison, so growing from 100,000 to 1,000,000 products raises the cost only from 17 to 20 comparisons. The iterative version is slightly better in pratice because it avoids the stack.

(d) Applying the Master Theorem

For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a):

- Case 1: f(n) = O(n^(log_b a − ε)) for some ε > 0, so T(n) = Θ(n^(log_b a)).
- Case 2: f(n) = Θ(n^(log_b a)), so T(n) = Θ(n^(log_b a) log n).
- Case 3: f(n) = Ω(n^(log_b a + ε)) for some ε > 0, plus the regularity condition, so T(n) = Θ(f(n)).

M4, current design: $T(n) = 4T(n/2) + n$

a	b	f(n)	n^(log_b a)
4	2	n	$n^{(\log_2 4)} = n^2$

$f(n) = n = O(n^{(2-\varepsilon)})$ with $\varepsilon = 1$, so Case 1 applies: $T(n) = \Theta(n^2)$.
(Exact solution: $T(n) = 2n^2 - n$.)

M4, proposed redesign: $T(n) = 2T(n/2) + n$

a b f(n) n^(log_b a)

2 2 n $n^{(\log_2 2)} = n$

$f(n) = n = \Theta(n)$, so Case 2 applies: $T(n) = \Theta(n \log n)$.
(Exact solution: $T(n) = n \log n + n$.)

What happens when the data doubles

	Growth	Effect of doubling n	Example, 1,000,000 → 2,000,000 (at 10^8 ops/s)
Current, $\Theta(n^2)$	$T(2n) \approx 4\,T(n)$	About 4 times slower	$2.0 \times 10^{12} \rightarrow 8.0 \times 10^{12}$ operations (5.6 hours → 22 hours)
Redesign, $\Theta(n \log n)$	$T(2n) = 2(1 + 1/\log n)\,T(n)$	A little over 2 times slower (about 2.1 at this size)	$2.09 \times 10^7 \rightarrow 4.39 \times 10^7$ operations (0.21 s → 0.44 s)

M5: $T(n) = 2T(n/2) + n \log n$

Here $a = 2$, $b = 2$, $f(n) = n \log n$ and $n^{(\log_b a)} = n$.

- Case 1? It needs f(n) to be polynomially smaller than n. But n log n is larger than n, so no.
- Case 2? It needs $f(n) = \Theta(n)$. But n log n grows faster than n, so no.
- Case 3? It needs $f(n) = \Omega(n^{(1+\varepsilon)})$ for some $\varepsilon > 0$. But log n grows slower than any n^{ε}, so n log n is not $\Omega(n^{(1+\varepsilon)})$. No.

None of the three standard cases applies. The function f(n) sits in the gap between Case 2 and Case 3.

Extended Master Theorem. If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$. Here $n^{\log_b a} = n$ and $f(n) = n \cdot \log^1 n$, so $k = 1$ and

$T(n) = \Theta(n \log^2 n)$

Recursion tree (confirming the result). Let $L = \log n$. At level i ($i = 0$ to $L - 1$) there are 2^i subproblems of size $n/2^i$, each costing $(n/2^i) \cdot \log(n/2^i)$. The cost of level i is

$2^i \cdot (n/2^i) \cdot \log(n/2^i) = n (L - i)$

Adding the levels, and then the n leaves of cost 1:

$T(n) = n \sum_{i=0}^{L-1} (L - i) + n = n \cdot L(L + 1)/2 + n = (n \log^2 n + n \log n)/2 + n$

This is $\Theta(n \log^2 n)$, matching the extended theorem. *Check:* $T(4) = 2T(2) + 4 \cdot 2 = 2 \cdot 4 + 8 = 16$, and $4 \cdot 2 \cdot 3/2 + 4 = 16$ ✓.

Meaning for ShopSphere. Re-sorting at every level costs an extra factor of log n compared with sorting once. For n = 1,000,000 that is about 2.1×10^8 operations, still only about 2 seconds, so it is acceptable (see part g).

(e) Best, Average and Worst Cases

Search version S1 (sequential scan, unsorted)

Case	Input condition	Comparisons
Best	The product is the first entry	1
Average	The product is equally likely to be at any position	$(n + 1)/2$ if it exists
Worst	The product is the last entry or is not in the catalogue	n

Average with probability p that the product exists. If it exists (probability p), it is at position i with probability 1/n, costing i comparisons. If it does not exist (probability $1 - p$), all n entries are checked.

$C_{avg}(n, p) = p \cdot (1/n)(1 + 2 + ... + n) + (1 - p) \cdot n$
$= p \cdot (n + 1)/2 + (1 - p) \cdot n$
$= n(1 - p/2) + p/2$

For $n = 1{,}000{,}000$ and $p = 0.5$:
$C_{avg} = 0.5 \times 500{,}000.5 + 0.5 \times 1{,}000{,}000 = 250{,}000.25 + 500{,}000 = 750{,}000.25$ comparisons

(Check: for $n = 1{,}000$, the formula gives 750.25, and a simulation of 200,000 searches gave 749.3.)

Search version S2 (binary search, sorted index)

Case	Input condition	Comparisons
Best	The key is exactly the middle entry	1
Average	A successful search with the key equally likely to be any entry	about $\log_2 n - 1$
Worst	The key is absent, or is at the deepest level of the search	$\lfloor \log_2 n \rfloor + 1$

At $n = 1{,}000{,}000$, the best case is 1, the average is about 18.9, and the worst case is 20. The average is close to the worst case because half of all entries sit at the deepest level of the search, and a quarter at the level above.

Comparison at $n = 1{,}000{,}000$, $p = 0.5$: S1 averages about 750,000 comparisons against about 20 for S2, so S2 is roughly 37,500 times cheaper per search.

(f) Proving an Asymptotic Bound

Definition. $f(n) = \Theta(g(n))$ means there exist positive constants c_1, c_2 and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

Take $f(n) = 3n^2 + 10n \log n + 50$ and $g(n) = n^2$.

Lower bound. For all $n \geq 1$, the terms $10n \log n$ and 50 are non-negative, so $f(n) \geq 3n^2$. Hence $c_1 = 3$.

Upper bound. For $n \geq 8$:

- $\log n \leq n$, so $10n \log n \leq 10n^2$

- $n^2 \geq 64 > 50$, so $50 \leq n^2$

Therefore $f(n) \leq 3n^2 + 10n^2 + n^2 = 14n^2$. Hence $c_2 = 14$ and $n_0 = 8$.

So $3n^2 \leq f(n) \leq 14n^2$ for all $n \geq 8$, which proves $f(n) = \Theta(n^2)$.

Check at n = 8: $f(8) = 3(64) + 10(8)(3) + 50 = 482$. The bounds are $3 \times 64 = 192$ and $14 \times 64 = 896$, and $192 \leq 482 \leq 896$ ✓. The code confirms the inequality for every n from 8 to 19,999.

f(n) is not O(n). Suppose, for contradiction, that there are constants $c > 0$ and n_1 such that $f(n) \leq c \cdot n$ for all $n \geq n_1$. Since $f(n) \geq 3n^2$, this would give $3n^2 \leq c \cdot n$, that is $n \leq c/3$, for all $n \geq n_1$. But that fails for any n larger than both n_1 and $c/3$. Contradiction, so f(n) is not O(n). (Equivalently, $f(n)/n \geq 3n \to \infty$.)

Meaning for ShopSphere. The quadratic term $3n^2$ dominates completely. For n = 1,000,000 it contributes 3×10^{12} operations, and the 10n log n term adds only about 2×10^8, which is under 0.01% of the total. Any speed-up of the report generator must therefore target the quadratic step.

(g) Scalability Decision

Operation counts (exact formulas from parts b to f; M5 uses the exact recursion-tree solution):

Module	Operations at n = 100,000	Operations at n = 1,000,000
M1 duplicate detector, n(n−1)/2	5.0×10^9	5.0×10^{11}
M2 API rate limiter, 2^⌈log n⌉ − 1	1.31×10^5	1.05×10^6
S2 binary search (one search)	17	20
M4 current, $2n^2 - n$	2.0×10^{10}	2.0×10^{12}
M4 redesign, n log n + n	1.76×10^6	2.09×10^7
M5 analytics, n log n (log n + 1)/2 + n	1.47×10^7	2.10×10^8
M5 report generator, f(n)	3.0×10^{10}	3.0×10^{12}

Running time at n = 1,000,000 (window = 10 minutes = 600 s):

Module	Time at 10^8 ops/s	Fits 10 min?	Time at 10^9 ops/s	Fits 10 min?
M1	5,000 s (83.3 min)	No	500 s (8.3 min)	Barely
M2	0.0105 s	Yes	0.001 s	Yes
S2 (one search)	0.0000002 s	Yes	0.00000002 s	Yes
M4 current	20,000 s (5.6 hours)	No	2,000 s (33.3 min)	No
M4 redesign	0.209 s	Yes	0.021 s	Yes
M5 analytics	2.1 s	Yes	0.21 s	Yes
M5 report generator	30,002 s (8.3 hours)	No	3,000 s (50 min)	No

A note on the missing exponent. M4 (current) and the report generator fail under either speed. M1 fails at 10^8 and passes at 10^9, but only just (8.3 of the 10 minutes), and the jobs share one nightly window. If they run one after another, M1 plus the other two heavy jobs would need over 5,000 seconds even at 10^9. So M1 should be redesigned under either assumption.

Today's picture (n = 100,000, at 10^8 ops/s): M1 takes 50 s, M4 current 200 s and the report generator 300 s. That is 550 s in total, just inside the 600 s window. The system works today only because it has not grown yet.

Modules that fail the 10-minute window

M1 (at 10^8), M4 (current design) and the M5 report generator.

Recommendations

Module	Decision	How, and why
M1	Redesign	Replace the all-pairs comparison. Put each product into a hash table keyed by (title, seller) and flag a product as a duplicate when its key is already there. This takes $\Theta(n)$ expected time (about 10^6 operations, roughly 0.01 s). A sort-and-scan alternative: sort by (title, seller) and compare neighbours, $\Theta(n \log n)$, about 2×10^7 operations. The trade-off is $\Theta(n)$ extra memory instead of $\Theta(1)$.

M4	Adopt the proposed redesign	Removing the replication changes $T(n) = 4T(n/2) + n$ ($\Theta(n^2)$) into $2T(n/2) + n$ ($\Theta(n \log n)$). That is about 2.09×10^7 operations instead of 2×10^{12}, roughly 95,000 times fewer, and doubling the data costs about 2.1 times instead of 4 times. The cost is losing the replicated partitions, so their fault-tolerance or availability role needs a cheaper substitute.
M5 report generator	Redesign	The $3n^2$ term must go. Find the quadratic step (for example a nested loop that compares or looks up records pairwise) and replace it with sorting, hashing or a single aggregation pass, giving $\Theta(n \log n)$. The remaining cost would be about $10n \log n \approx 2 \times 10^8$ operations, around 2 seconds. This estimate assumes the quadratic step can be replaced, which depends on what the report computes.
M5 analytics	Keep, optional improvement	It already fits (about 2.1 s). If wanted, sort once and merge by revenue at each level instead of re-sorting. That gives $T(n) = 2T(n/2) + n = \Theta(n \log n)$, about 10 times fewer operations.
M2	Keep	$\Theta(n)$, about a million operations. The claim that it is $\Theta(n \log n)$ was wrong, so it needs no redesign.
M3 (S2)	Use S2	About 20 comparisons per search against about 750,000 for S1, and S2 is the only version that scales. Prefer the iterative form to save stack space. The cost is keeping the index sorted, which needs maintenance when products are added.

Overall conclusion

Three modules, M1, M4 (current) and the report generator, have quadratic cost. They are the only ones that fail, and they get worse by a factor of 100 when the catalogue grows tenfold. The other modules scale linearly or better and pass easily. Redesigning the three quadratic modules to $\Theta(n)$ or $\Theta(n \log n)$ brings each well inside the 10-minute window.